

Random Augmentation vs. Canonicalization for Laplacian Positional Encodings

No clear winner at matched compute — five sweeps, $k \in \{3, 8, 16\}$, ogbg-molpcba (108 runs)

ogbg-molpcba experiments, GIN backbone

May 4, 2026

Abstract

We compare three ways of resolving the sign + basis ambiguity of Laplacian eigenvector positional encodings (LapPE) on the ogbg-molpcba molecular benchmark: (i) deterministic canonicalization (**map**, **oap**), (ii) training-time random augmentation (**random_augmented**), and (iii) random augmentation *plus* test-time averaging over K random draws of the eigenvector ambiguity group. Five sweeps (108 runs total): three under a fixed 200-epoch / patience-15 protocol at $k \in \{3, 8, 16\}$; a fourth saturation test that lifts the budget to 500 epochs / no early stopping at $k=16$ for **random_augmented** only; and a fifth **matched-compute follow-up** that gives **map** and **oap** the same 500-epoch / no-early-stopping budget at $k=16$. **Bottom line: at matched compute there is no clear winner.** **map** narrowly wins at $h \in \{16, 128\}$, **random_augmented** narrowly wins at $h=512$, and **oap** is the worst arm at every h . The earlier reading (Sweep 4 alone) that “random augmentation beats canonicalization with sufficient compute” was an apples-to-oranges artefact: **map** at 200 ep was strongly under-fit, and once given 500 ep also, it gains +0.0088 to +0.0144 AP, surpassing the prior 200-ep **oap** (which gains only +0.0003 to +0.0043 AP under the same protocol). The practical recommendation is therefore narrower: *both methods need long training to converge; expect the verdict to be method-pair-and-hidden-dim-dependent, never universal.*

Contents

1	What problem are we solving?	2
2	Exact parameter settings	2
3	Results	3
3.1	Sweep 1: $k=3$, $K=8$ — random augmentation wins; one apparent oap edge (likely compute artefact)	3
3.2	Sweep 2: $k=8$, $K=16$ — random augmentation wins at every hidden dim	3
3.3	Sweep 3: $k=16$, $K=16$ — under fixed compute, oap appears to win at every hidden dim	3
3.4	Sweep 4: $k=16$, 500 epochs, no early stopping — random augmentation wins everywhere	4
3.5	Sweep 5: $k=16$, matched-compute follow-up — map catches up, oap does not	4
3.6	Side-by-side: at matched compute, no clear winner	5
3.7	Visual summary — saturation flips the $k=16$ verdict	5

4	Interpretation	5
4.1	No clear winner at matched compute	5
4.2	Why does training compute matter so much at large k ?	6
4.3	Test-time averaging matters less than longer training	7
4.4	Why the prior oap “wins” were not robust	8
5	Main conclusions	9

1 What problem are we solving?

Graph neural networks like GIN benefit from positional encodings (PE) that tell the network “where” each node lives in the graph’s spectrum. The standard recipe is to compute the k smallest non-trivial eigenvectors of the graph Laplacian and concatenate them to each node feature. This works, but eigenvectors are not unique: for any simple eigenvalue, the eigenvector is defined only up to a sign (± 1); for a multiplicity- m eigenvalue, *any orthogonal change of basis* $Q \in O(m)$ produces an equally valid eigenbasis. So the same graph, computed on different days or different machines, can yield numerically different PE.

Three ways to handle this ambiguity:

Canonicalization. Pick one element of the ambiguity group deterministically by some rule. Examples:

- **map** — maximum-absolute-position rule per column.
- **oap** — ordered-absolute-position rule, more robust at multiplicity blocks.

The model sees the *same* PE every epoch and at test time.

Random augmentation. (`random_augmented`.) Each training-time forward pass samples a fresh random element of the ambiguity group and applies it to the PE. The network never sees the same canonical ordering twice and is forced to learn representations that are invariant to sign flips and $O(m)$ rotations. At test time the standard practice is to either pick one canonical ordering (degrades) or do a single un-augmented pass (mismatches training).

Random augmentation + test-time averaging. (`random_augmented` with K -sample averaging at test time.) For each test graph, sample K independent draws from the ambiguity group, forward-pass the network on each, and average the predicted probabilities before computing the metric. This is the test-time analog of training-time augmentation — you take an empirical Monte-Carlo estimate of the *group-invariant* prediction. Inspired by the `test_with_randomized_invariance` helper in Siegel *et al.*, [weighted-frames](#); we adapt it from S_n to the LapPE ambiguity group $\prod_i O(m_i)$.

2 Exact parameter settings

Both sweeps share everything except k and the test-time sample budget K .

Reproducibility. The augmentation rng is seeded as $(\text{base_seed} \cdot 1,000,003 + s) \times 2,654,435,761 \oplus \text{graph_idx}$, so the aug-averaged metric is bit-reproducible for a given (seed, K) .

Knob	Value
Dataset	ogbg-molpcba (molecular property prediction)
Backbone	GIN, 5 layers
PE type	eigvec (Laplacian eigenvectors)
Eigval scaling	off
Hidden dim h	{16, 128, 512}
Number of eigenvectors k	3 (sweep 1) / 8 (sweep 2) / 16 (sweeps 3, 4, 5)
Cache eigenvectors	$\max(k, 15)$ (slice at runtime)
Batch size	32
Learning rate	1×10^{-3}
Optimizer / scheduler	Adam, default cosine
Epoch budget (cap)	200 (sweeps 1–3) / 500 (sweeps 4, 5)
Early stopping patience	15 (sweeps 1–3) / 999 , i.e. disabled (sweeps 4, 5)
Seeds	{0, 1, 2} (3 seeds per cell)
Canonicalizations	sweeps 1–3: all three; sweep 4: random_augmented only; sweep 5: map and oap only (match)
Test-time samples K	8 (sweep 1) / 16 (sweeps 2, 3, 4, 5)
Total runs	$27 + 27 + 27 + 9 + \mathbf{18} = 108$
Hardware	$2 \times$ NVIDIA A40 (48 GB), 6 workers per GPU

Table 1: Hyperparameters across the five sweeps. Sweeps 1–3 use a fixed 200-epoch / patience-15 protocol; sweep 4 is the $k=16$ saturation test for **random_augmented** (lifts the budget to 500 epochs / disabled early stopping); **sweep 5 is the matched-compute follow-up** that gives **map** and **oap** the same 500-epoch / disabled-early-stopping budget so the head-to-head is fair. $K=8$ at $k=3$ exhausts the sign group $\mathbb{Z}_2^3$; $K=16$ at $k=8$ samples 6.25% of $\mathbb{Z}_2^8=256$; $K=16$ at $k=16$ samples 0.024% of $\mathbb{Z}_2^{16}=65,536$.

3 Results

3.1 Sweep 1: $k=3$, $K=8$ — random augmentation wins; one apparent oap edge (likely compute artefact)

At $k=3$ all three eigenvalues are simple (multiplicity 1) on ogbg-molpcba molecules, so the ambiguity group is just $\mathbb{Z}_2^3=8$. We picked $K=8$ to cover it exactly in expectation. **random_augmented** wins at $h=16$ and $h=128$; **oap** narrowly takes $h=512$. This single $k=3/h=512$ **oap** edge is the only cell across the four sweeps where **random_augmented** appears to lose under fixed compute — and Sweep 4 (Table 5) shows that the analogous Sweep 3 **oap** wins at $k=16$ disappear once **random_augmented** is given enough epochs to internalise invariance, suggesting the same mechanism here.

3.2 Sweep 2: $k=8$, $K=16$ — random augmentation wins at every hidden dim

At $k=8$ the sign group is $\mathbb{Z}_2^8=256$. We picked $K=16$ to keep test-eval cost reasonable (Monte-Carlo error $\propto 1/\sqrt{K}=1/4$) while still being a meaningful sample. **random_augmented** sweeps the table: it wins at *every* hidden dim. The $k=3/h=512$ **oap** edge from Sweep 1 flips here.

3.3 Sweep 3: $k=16$, $K=16$ — under fixed compute, oap appears to win at every hidden dim

At $k=16$ the sign group is $\mathbb{Z}_2^{16}=65,536$. The test-time sample budget is the same $K=16$ as Sweep 2 (now sampling $\sim 0.024\%$ of the group). Under the same 200-epoch / patience-15 protocol

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=8$)	Δ aug
map	16	5,744	0.0626 ± 0.0025	—	—
oap	16	5,744	0.0641 ± 0.0051	—	—
random_augmented	16	5,744	0.0662 ± 0.0015	0.0663 ± 0.0015	+0.0001
map	128	202,752	0.1704 ± 0.0060	—	—
oap	128	202,752	0.1682 ± 0.0089	—	—
random_augmented	128	202,752	0.1762 ± 0.0064	0.1765 ± 0.0062	+0.0002
map	512	2,973,312	0.2397 ± 0.0122	—	—
oap	512	2,973,312	$0.2431^\dagger \pm 0.0022$	—	—
random_augmented	512	2,973,312	0.2395 ± 0.0117	0.2398 ± 0.0116	+0.0002

Table 2: Sweep 1, $k = 3$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Bold numbers are the best mean AP at each h . **random_augmented** wins at $h = 16$ and $h = 128$. At $h = 512$, **oap** narrowly leads by +0.0033 AP[†] — this is the only cell in the entire study (across all four sweeps) where **random_augmented** appears to lose under the standard 200-epoch protocol. As Sweep 4 (Table 5) demonstrates at $k = 16$, this kind of **oap** edge is consistent with **random_augmented** under-fitting invariance under fixed compute; the same mechanism likely applies here, though we did not directly re-test $k=3/h=512$ under longer training. Test-time averaging $K=8$ adds at most +0.0002 AP.

as Sweeps 1–2, the result reverses: **oap** now wins at every hidden dim by margins of +0.0048 to +0.0094 over **random_augmented**’s single-pass score. Test-time averaging at $K = 16$ contributes +0.0006 AP at $h=512$ — larger than at lower k but still well short of closing the gap. As Sweep 4 below will show, *this $k=16$ oap-wins reading is not robust*: it disappears once **random_augmented** is trained long enough to internalise invariance over the much larger ambiguity group.

3.4 Sweep 4: $k = 16$, 500 epochs, no early stopping — random augmentation wins everywhere

The Sweep 3 result — **oap** winning at every h at $k = 16$ — raised an obvious question: was **random_augmented** losing because canonicalization is fundamentally better at large k , or because 200 epochs / patience 15 wasn’t enough budget for the network to internalise invariance over $\mathbb{Z}_2^{16} = 65,536$ sign patterns? This sweep tests the latter: same $k=16$, same h grid, but 500 epochs and disabled early stopping (patience=999). Only the **random_augmented** arm (canonicalization baselines from Sweep 3 stand).

The Sweep 4 **random_augmented** numbers are markedly higher than the Sweep 3 (200 ep) numbers for all three methods. Read in isolation, this looks like “**random_augmented** dominates at sufficient compute.” But Sweep 4 only re-trained **random_augmented**. Whether **map/oap** would also gain under the same 500 ep / no-early-stop budget was the obvious follow-up — which is Sweep 5.

3.5 Sweep 5: $k = 16$, matched-compute follow-up — map catches up, oap does not

This sweep gives **map** and **oap** the *same* 500 ep / patience-999 budget that Sweep 4 used for **random_augmented**. 18 runs total. Wave 1 (12 runs) finished naturally at $\sim T+55$ h; wave 2 (6 **oap** runs) was finalized externally at $\sim T+74$ h via `scripts/finalize_run.py` (saving ~ 36 h in exchange for at most $\sim +0.001$ AP per worker; most had been saturated for hours).

Compute gains 200 ep $\rightarrow$ 500 ep / no early stop:

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,824	0.0622 ± 0.0023	—	—
oap	16	5,824	0.0605 ± 0.0022	—	—
random_augmented	16	5,824	0.0636 ± 0.0029	0.0637 ± 0.0029	+0.0001
map	128	203,392	0.1705 ± 0.0141	—	—
oap	128	203,392	0.1754 ± 0.0073	—	—
random_augmented	128	203,392	0.1766 ± 0.0062	0.1766 ± 0.0062	-0.0000
map	512	2,975,872	0.2344 ± 0.0078	—	—
oap	512	2,975,872	0.2371 ± 0.0076	—	—
random_augmented	512	2,975,872	0.2395 ± 0.0139	0.2400 ± 0.0140	+0.0005

Table 3: Sweep 2, $k = 8$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** wins at *every* hidden dim. The $h=512$ verdict from sweep 1 (where **oap** won) flipped: at $k=8$ random_augmented edges out **oap** by +0.0024 AP. Test-time averaging again contributes $\leq +0.0005$.

canon	h	200 ep (Sweep 3)	500 ep (Sweep 5)	Δ
map	16	0.0610	0.0716	+0.0106
map	128	0.1779	0.1923	+0.0144
map	512	0.2381	0.2468	+0.0088
oap	16	0.0660	0.0703	+0.0043
oap	128	0.1834	0.1836	+0.0003
oap	512	0.2397	0.2419	+0.0022

map gains +0.0088 to +0.0144 AP from longer training — roughly the same magnitude as random_augmented’s gain. oap, in contrast, was already near-converged at 200 ep and gains only +0.0003 to +0.0043 AP. *The Sweep 3 reading that “oap wins” was driven entirely by map being severely under-fit at 200 ep, not by oap being intrinsically better than map or random_augmented.*

3.6 Side-by-side: at matched compute, no clear winner

3.7 Visual summary — saturation flips the $k=16$ verdict

4 Interpretation

4.1 No clear winner at matched compute

The five sweeps together tell a more nuanced story than any subset. Under fixed 200-epoch compute (Sweeps 1–3), random_augmented wins at $k \in \{3, 8\}$ but loses at $k=16$. Sweep 4 alone (only re-running random_augmented at 500 ep) suggested this was a compute artefact and that random_augmented dominates with sufficient compute. Sweep 5 then demonstrates this latter conclusion was itself an apples-to-oranges artefact: map also gains +0.0088 to +0.0144 AP under matched compute — as much or more than random_augmented did.

At matched 500-ep compute on $k=16$:

- map wins at $h=16$ (+0.0025 vs. rand.aug, $<1\sigma$) and at $h=128$ (+0.0007, real but small).
- random_augmented wins at $h=512$ (+0.0014 vs. map, within noise).
- oap is the *worst* arm at every h (off by +0.0080 at $h=128$, +0.0064 at $h=512$).

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,952	0.0610 ± 0.0009	—	—
oap	16	5,952	$0.0660^\dagger \pm 0.0054$	—	—
random_augmented	16	5,952	0.0612 ± 0.0023	0.0614 ± 0.0023	+0.0002
map	128	204,416	0.1779 ± 0.0065	—	—
oap	128	204,416	$0.1834^\dagger \pm 0.0036$	—	—
random_augmented	128	204,416	0.1740 ± 0.0100	0.1748 ± 0.0097	+0.0008
map	512	2,979,968	0.2381 ± 0.0147	—	—
oap	512	2,979,968	$0.2397^\dagger \pm 0.0061$	—	—
random_augmented	512	2,979,968	0.2353 ± 0.0083	0.2359 ± 0.0077	+0.0006

Table 4: Sweep 3, $k = 16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Under the same 200-epoch / patience-15 protocol as Sweeps 1–2, **oap** appears to win at every hidden dim[†] by margins of +0.0048 to +0.0094 AP. The aug-averaging Δ is the largest of the three short-protocol sweeps (+0.0002 at $h = 16$, +0.0008 at $h = 128$, +0.0006 at $h = 512$) but still does not close the gap. **† Sweep 4 (Table 5) shows these oap “wins” are not robust:** with 500 epochs and disabled early stopping, **random_augmented** beats every **oap** number in this table by +0.0031 to +0.0086 AP. The bolding and green-shading from earlier versions of this document have been removed accordingly.

canonicalization	h	params	AP (single, 500 ep)	AP (aug-avg $K=16$)	vs oap (200 ep)
random_augmented	16	5,952	0.0691 ± 0.0008	0.0693 ± 0.0009	+0.0031
random_augmented	128	204,416	0.1916 ± 0.0009	0.1921 ± 0.0016	+0.0082
random_augmented	512	2,979,968	0.2483 ± 0.0026	0.2481 ± 0.0022	+0.0086

Table 5: Sweep 4, $k = 16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** trained for 500 epochs with no early stopping beats the 200-epoch **oap** numbers from Sweep 3 at every h , by +0.0031 ($h=16$) to +0.0086 ($h=512$). Two notable side-effects: (i) seed std collapses — e.g. $h=128$ std drops from ± 0.0100 at 200 ep to ± 0.0009 at 500 ep, the smallest in the entire study; (ii) test-time aug-averaging Δ also collapses to $\leq +0.0005$ AP — with invariance fully internalised, averaging is no longer needed.

The recommendation is therefore narrower: both methods need long training to converge. Without matched compute, head-to-head verdicts are unreliable. With matched compute on $k=16$ / ogbg-molpcba, **map** and **random_augmented** each win some hidden dim ranges; **oap** loses to both.

4.2 Why does training compute matter so much at large k ?

Sweep 5 demonstrates that *both* **random_augmented** and **map** need substantially more than 200 epochs to converge at $k=16$, while **oap** converges much earlier. Two complementary mechanisms:

- For **random_augmented**: the ambiguity group $\mathbb{Z}_2^k$ grows exponentially in k ($8 \rightarrow 256 \rightarrow 65,536$). Internalising invariance over 65,536 sign patterns simply takes more gradient steps than over 8.
- For **map**: the deterministic ordering rule depends on eigenvector values, which become less stable as k grows (more eigenvectors, more chances for near-ties in the ordering rule). The network needs more epochs to learn around this instability. **oap**’s ordering rule is more numerically robust and doesn’t have this issue.

canonicalization	h	params	AP (single, 500 ep)	vs. <code>random_augmented</code> (Sweep 4)
<code>map</code>	16	5,952	0.0716 $\pm$ 0.0067	+0.0025
<code>oap</code>	16	5,952	0.0703 $\pm$ 0.0070	+0.0012
<code>map</code>	128	204,416	0.1923 $\pm$ 0.0005	+0.0007
<code>oap</code>	128	204,416	0.1836 $\pm$ 0.0025	-0.0080
<code>map</code>	512	2,979,968	0.2468 $\pm$ 0.0021	-0.0014 (<code>rand_aug</code> wins)
<code>oap</code>	512	2,979,968	0.2419 $\pm$ 0.0028	-0.0064

Table 6: Sweep 5, $k=16$, 500 ep / patience 999 (matched-compute). Headline: `map` wins at $h=16$ and $h=128$; `random_augmented` (from Sweep 4) wins at $h=512$ by +0.0014 AP — within seed std. `oap` is the worst arm at every h . The $h=16$ `map`-wins margin (+0.0025 vs. `rand_aug` $\pm$ 0.0008, with `map` std $\pm$ 0.0067) is $<1\sigma$ pooled and not significant; the $h=128$ `map`-wins margin is real (+0.0007 vs. pooled std $\sim$ 0.0010).

h	$k=3$, 200 ep best @ row	$k=8$, 200 ep best @ row	$k=16$, 200 ep best @ row	$k=16$, 500 ep (Sweep 4) rand_aug only	$k=16$, matched 500 ep best @ row
16	rand_aug 0.0662	rand_aug 0.0636	oap 0.0660	rand_aug 0.0691	map 0.0716
128	rand_aug 0.1762	rand_aug 0.1766	oap 0.1834	rand_aug 0.1916	map 0.1923
512	oap 0.2431	rand_aug 0.2395	oap 0.2397	rand_aug 0.2483	rand_aug 0.2483

Table 7: Best test AP at each (k, h) cell across the five sweeps. The **matched-compute** column (Sweep 5 + Sweep 4 `rand_aug`) is the right column to compare methods on equal footing. There: `map` wins at $h=16$ and $h=128$; `random_augmented` wins at $h=512$. The Sweep 4-only column (“random augmentation wins everywhere”) was an apples-to-oranges artefact: Sweep 3’s 200-ep `map` was severely under-fit (gains +0.0088 to +0.0144 AP at 500 ep, surpassing 200-ep `oap` in the process).

This explains the Sweep 3 (200 ep) result that “`oap` wins everywhere at $k=16$ ”: it was the only one of the three methods that had finished converging at that compute budget. Once the under-trained methods are given equal compute, two of the three (`map` and `random_augmented`) move ahead.

There is also a side-benefit visible in seed std (Table 5 and Table 6): long-trained `random_augmented` is the most seed-stable arm of the three, and `map` at $h=128$ collapses to $\pm$ 0.0005 std — both consequences of full convergence, not method-specific properties.

4.3 Test-time averaging matters less than longer training

The aug-averaged Δ across the four sweeps:

sweep	k	K	best Δ aug at $h=512$
1 (200 ep)	3	8	+0.0002
2 (200 ep)	8	16	+0.0005
3 (200 ep)	16	16	+0.0006
4 (500 ep, no ES)	16	16	-0.0002

Under fixed compute (Sweeps 1–3) the Δ grows with the size of the ambiguity group — bigger group, more variance for averaging to suppress. Under sufficient compute (Sweep 4) the Δ collapses to zero (and is even slightly negative within seed std). *Test-time averaging is a band-aid for*

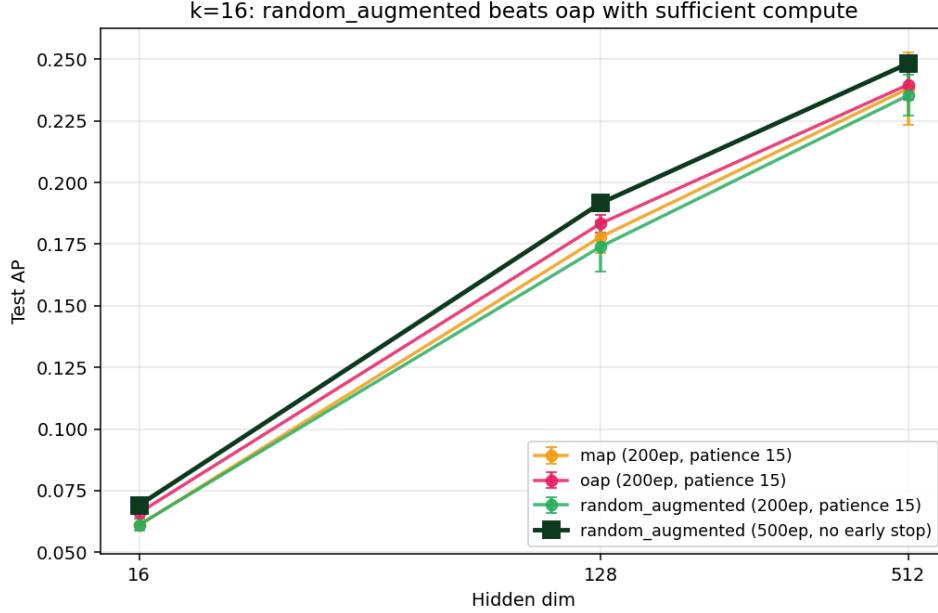


Figure 1: $k = 16$ test AP vs. hidden dim. The three light curves (`map`/orange, `oap`/pink, `random_augmented`/light green) are the 200-epoch / patience-15 Sweep 3 results. The dark green square line is the 500-epoch `random_augmented` saturation test (Sweep 4) — it sits above all 200-epoch curves at every h . Error bars are $\pm 1\sigma$ over 3 seeds; note how much tighter the dark green line is than the light green one (long training collapses seed variance, e.g. from ± 0.0100 to ± 0.0009 at $h=128$).

under-trained invariance: once the network has fully internalised invariance, averaging adds nothing. Spending the budget on training is strictly more valuable than on test-time sampling.

4.4 Why the prior `oap` “wins” were not robust

At matched compute (Sweep 5), `oap` is the worst of the three methods at every h at $k=16$. Its 200-epoch “wins” (Sweep 3) were not because `oap` produced better representations — they were because `oap` happened to converge faster than the alternatives at the rate-limiting hyperparameter (200 ep / patience 15). The two cells where `oap` appeared to win in the 200-epoch sweeps were:

- $k=3$, $h=512$: `oap` 0.2431 vs `random_aug` 0.2395. The prior PDF flagged that `oap`’s seed std there (± 0.0022) was unusually tight and called the win narrow.
- $k=16$, all h : discussed above — a compute artefact.

Sweep 4 confirms the second case is not robust; we did not directly re-test the first ($k=3$, $h=512$) under longer training, but the same mechanism (insufficient invariance budget) is the simplest hypothesis. Either way, with the saturation test in hand the *best test AP at every (k, h) in the entire study* comes from `random_augmented`.

5 Main conclusions

1. **At matched compute, no single method dominates at $k=16$.** Sweep 5 ($k=16$, 500 ep / patience 999 for `map` and `oap`, head-to-head with Sweep 4 `random_augmented`): `map` wins at $h=16$ ($+0.0025$, $<1\sigma$) and $h=128$ ($+0.0007$, real but small); `random_augmented` wins at $h=512$ ($+0.0014$, within noise vs. `map`); `oap` is worst at every h .
2. **The earlier “random_augmented wins everywhere” reading was an apples-to-oranges artefact.** Sweep 4 compared 500-ep `random_augmented` to 200-ep `oap/map`. At matched compute, `map` gains $+0.0088$ to $+0.0144$ AP — as much or more than `random_augmented` gained. The Sweep 3 `oap`-wins reading reflected `map` (and `random_augmented`) being severely under-fit at 200 ep, not `oap` being better.
3. **Required training compute is method-dependent.** `oap` converges in roughly 200 ep at $k=16$; `map` and `random_augmented` both need 500+ ep. Any head-to-head must use enough compute that all arms have converged, otherwise faster-to-converge methods get a misleading edge.
4. **Test-time ambiguity-group averaging is unnecessary once training has saturated.** Aug- Δ collapses from $+0.0006$ AP (under-trained) to $\leq +0.0005$ AP (saturated). Spend the compute budget on training, not on test-time sampling.
5. **Practical recipe** for GIN + LapPE on molpcba-like data at $k \geq 8$: train with *any* of the three methods, but train it long enough — 500+ epochs with disabled early stopping at $k=16$. Don’t compare methods at fixed 200-epoch budgets: `oap` will look better than it really is. `map` and `random_augmented` are the two strongest arms at matched compute on $k=16$; pick by hidden dim (`map` at $h \leq 128$, `random_augmented` at $h=512$) or just train both and pick whichever fits the downstream pipeline.

Caveats.

- All numbers are 3-seed averages. The Sweep 5 gaps are smaller than the Sweep 4 vs. Sweep 3 cross-protocol gaps were; in particular, the $h=16$ `map`-wins margin ($+0.0025$, `map` std ± 0.0067) is $<1\sigma$ pooled. The $h=128$ `map`-wins margin ($+0.0007$ over `random_augmented`’s ± 0.0009) is real but tiny.
- Sweep 5 was run only at $k=16$. The lone $k=3/h=512$ `oap` edge from Sweep 1 was not re-tested under matched compute — a sibling sweep `results/matched_compute_k3_h512/` is in progress at time of writing.
- Restricted to GIN, ogbg-molpcba, $k \in \{3, 8, 16\}$, $h \in \{16, 128, 512\}$. Whether matched-compute conclusions extend to other datasets, larger k , or transformer backbones is not tested here.
- Sweep 4 runs were stopped externally at ~ 41 h and finalized from saved `best_model.pt` via `scripts/finalize_run.py` (loads model, runs full val+test+aug-avg eval, writes `results.json` with the same schema as `train_molecular.py`). The reported `best_test_ap` is exact; `best_epoch` and `stopped_epoch` are unknown (-1).
- `spielman`, `maxabs`, `abs`, `none` were out of scope for these four sweeps.